

Grundlagen der C-Programmierung

Dr. Henrik Brosenne
Georg-August Universität Göttingen
Institut für Informatik

Wintersemester 2024/25

Einleitung

Programmiersprachen

Schritte der Programmentwicklung

Der Zeichensatz von C

Der Aufbau von C-Quellcode

Die Geschichte von C

Hello World

Programmiersprache

Maschinencode ist die für den Prozessor eines Computers erforderliche Darstellung von Befehlen, sodass diese unmittelbar ausgeführt werden können.

Ein **ausführbares Programm** enthält Maschinencode und Daten.

Eine **Programmiersprache** ist eine formale (exakte künstliche) Sprache zur Formulierung von **Quellcode** (Quellcode–Programmen).

Die Definition einer Programmiersprache umfasst

- **Syntax**, Aufbau gültiger Sätze.
- **Semantik**, Bedeutung gültiger Sätze.

Ein gültiger Quellcode kann mit Hilfe spezieller Werkzeuge weiterverarbeitet, z.B. in ein ausführbares Programm übersetzt, werden.

Programmiersprache C

C ist eine Programmiersprache.

Syntax und Semantik werden von einem internationalen Standard festgelegt, in diesem Kurs ISO/IEC 9899:2011 oder kurz **C11**.

Ein Quellcode in C kann in ein ausführbares Programm übersetzt werden.

Das Übersetzen findet in mehreren Schritten, mit folgenden Werkzeugen, statt.

- Der **Compiler** erzeugt aus dem Quellcode einen **Objektcode**. Dabei werden Syntax und Semantik überprüft.
- Der **Linker** bindet Objektcode (möglicherweise mehreren als einen) mit den nötigen Teilen der **Standardbibliothek** zu einem ausführbaren Programm zusammen, dabei erfolgen Konformitätsprüfungen, z.B. ob eine im Objektcode benutzte Komponente im fertigen Programm auch verfügbar sein wird.

Der Standard legt fest welche bereits fertig compilierte Funktionen, Konstanten, Datenstrukturen etc. in der Standardbibliothek enthalten sind.

Das erste Programm

Der Quellcode ist in der Datei `hello.c` gespeichert.

```
1  #include <stdio.h>
2
3  int main(void) {
4      printf("Hello_World!\n");
5      return 0;
6  }
```

Compilieren

```
> gcc -c hello.c
```

Binden

```
> gcc -o hello hello.o
```

Ausführen

```
> ./hello
Hello World!
```

Compilieren und Binden

Compilieren und Binden kann von `gcc` (*GNU Compiler Collection*) auch mit einem Aufruf angefordert werden.

```
> gcc -o hello hello.c
> ./hello
Hello World!
```

Am Ablauf ändert das nichts, es werden verschiedene Schritte hintereinander ausgeführt.

Der Schalter `-o` ist optional, man kann damit den Namen des ausführbaren Programms festlegen. Ohne diesen Schalter heißt es `a.out`

```
> gcc hello.c
> ./a.out
Hello World!
```

Einleitung

Programmiersprachen

Schritte der Programmentwicklung

Der Zeichensatz von C

Der Aufbau von C-Quellcode

Die Geschichte von C

Hello World

Programmentwicklung mit C (1/3)

❶ Formulierung der **Aufgabe**.

- ▶ Welche Daten stehen in welcher Form zur Verfügung?
- ▶ Welche Resultate möchte man haben?

❷ Entwicklung eines **Algorithmus** zur Lösung der Aufgabe.

Es gibt keine allgemeinen Regeln, wie man zu einem Problem einen geeigneten Algorithmus.

Die Formulierung von Algorithmen erfolgt abseits vom Rechner.

❸ **Codierung** des Algorithmus.

Die Einzelschritte des Algorithmus werden in Anweisungen von C übertragen in den Rechner eingegeben und dort gespeichert (z.B. auf der Festplatte). So entsteht der **Quellcode** bzw. **Quelltext** eines Programms.

Ein **Editor** ist ein spezielles Programme für diese Aufgabe, dass den Entwickler durch automatische Formatierung des Quelltextes, Syntaxeinfärbung, etc. unterstützen kann.

4 Übersetzen des Quellcodes mit einem **Compiler**.

In einem neu erstellten Quellcode wird der Compiler i.a. mehr oder minder viele formale Fehler finden – diese müssen zunächst korrigiert werden.

Die Folge editieren/übersetzen wird so lange wiederholt werden, bis das Programm keine formalen Fehler mehr enthält.

Wenn der Compiler keine Fehler entdeckt, erzeugt er **Objektcode**.

5 Binden des Objektcodes mit dem **Linker**.

Der Linker fügt Objektcode aus verschiedenen Quellen zusammen, z.B. den selbst übersetzten Objektcode mit den benötigten Routinen aus der Standardbibliothek.

Auch dabei können Fehler auftreten, z.B. dass der Linker benötigte Routinen nicht findet.

Wenn der Linker keine Fehler entdeckt, erzeugt er ein **ausführbares Programm**.

6 **Ausführen** des fertigen Programms.

Man kann nicht davon ausgehen, dass jedes Programm beim ersten Versuch korrekt arbeitet.

Logischen Fehler, z.B. bei der Codierung des Algorithmus, müssen gefunden und korrigiert werden.

Einleitung

Programmiersprachen

Schritte der Programmentwicklung

Der Zeichensatz von C

Der Aufbau von C-Quellcode

Die Geschichte von C

Hello World

Graphische Zeichen

Zeichen, die in einer Quelltextdatei vorkommen dürfen.

Der Zeichensatz ANSI-C umfasst 91 graphische Zeichen.

- 26 Großbuchstaben des (englischen) Alphabets

A	B	C	D	E	F	G	H	I	J	K	L	M
N	O	P	Q	R	S	T	U	V	W	X	Y	Z

- 26 Kleinbuchstaben des (englischen) Alphabets

a	b	c	d	e	f	g	h	i	j	k	l	m
n	o	p	q	r	s	t	u	v	w	x	y	z

- 10 Ziffern

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

- 29 Sonderzeichen

!	"	#	%	&	'	(	)	*	+	,	-	.	/	:
;	<	=	>	?	[	\	]	^	_	{		}	~	

Nicht graphische Zeichen (*white spaces*)

Die nicht graphische Zeichen, die in einer Quelltextdatei vorkommen dürfen, werden kollektiv als **white spaces** bezeichnet.

- Leerzeichen ()
- Tabulator-Zeichen (horizontal und vertikal)
- Seitenvorschub-Zeichen
- Zeilenende-Kennzeichnung.

Die Zeilenende-Kennzeichnung kann implementationsabhängig aus mehreren Zeichen bestehen.

- UNIX/Linux-Textdateien verwenden nur das *line-feed*-Zeichen als Zeilenendezeichen.
- DOS/Windows-Textdateien benutzen dagegen eine Kombination aus *line-feed* und *carriage-return*.
- Apple-Textdateien wiederum benutzen nur das *carriage-return*.

Verarbeitbare Zeichen

C-Programme können jedoch während der Ausführung auch andere Zeichen **verarbeiten**.

Eine Benutzereingabe kann also durchaus auch die deutschen Umlaute oder das ß enthalten, obwohl der Quelltext diese Zeichen nicht enthalten darf.

Zur Darstellung dieser Zeichen im Quelltext z.B. in Zeichenkettenkonstanten gibt es spezielle Zeichenkombinationen, die sogenannten **Escape-Sequenzen**.

Einleitung

Programmiersprachen

Schritte der Programmentwicklung

Der Zeichensatz von C

Der Aufbau von C-Quellcode

Die Geschichte von C

Hello World

Aufbau von C-Quellcode

Ein C-Quellcode besteht, formal betrachtet, aus einer Ansammlung von **Funktionen**.

Genau eine dieser Funktionen ist ausgezeichnet durch den Namen `main` und realisiert das **Hauptprogramm**.

Alle anderen Funktionen sind **Unterprogramme** und müssen andere Namen tragen. Das kleinste Programm besteht nur aus der Funktion `main`. Fehlt sie, gibt der Linker eine Fehlermeldung aus.

Alle Funktionen sind formal völlig gleichberechtigt. Insbesondere heißt das, dass zwar alle Funktionen in beliebiger Reihenfolge angegeben werden können, nicht jedoch ineinander geschachtelt.

Schema von C-Quellcode

Die Ausführung des Programms beginnt am Anfang des Hauptprogramms `main`.

Die Funktionen `funkt1`, ..., `funktN` werden jeweils aufgerufen

- vom Hauptprogramm und/oder
- von einer anderen Funktion und/oder
- von sich selbst (**Rekursion**).

Direktiven für den Präprozessor
globale Deklarationen

```
... funkt1(...) {  
    lokale Deklarationen  
    Anweisungsfolge  
}
```

```
...
```

```
... funktN(...) {  
    lokale Deklarationen  
    Anweisungsfolge  
}
```

```
int main(...) {  
    lokale Deklarationen  
    Anweisungsfolge  
}
```

Funktionen und Prozeduren

Mit Funktionen sind Unterprogramme gemeint, die nach dem Aufruf einen Funktionswert zurückgeben während Prozeduren keinen Wert zurückgeben und allein durch ihre **Nebeneffekte** wirken.

Formal unterscheidet C bei Unterprogrammen nicht zwischen Prozeduren und Funktionen. Allerdings kennt C diese Unterscheidung in der Sache durchaus.

Einleitung

Programmiersprachen

Schritte der Programmentwicklung

Der Zeichensatz von C

Der Aufbau von C-Quellcode

Die Geschichte von C

Hello World

Geschichte (1/2)

1972 entwickelte Dennis M. Ritchie die Programmiersprache C im Zusammenhang mit einer Implementation des Betriebssystems UNIX. Grundlage war B, eine Weiterentwicklung von BCPL (*Basic Combined Programming Language*), das zu C erweitert wurde.

1977 erfolgte die erste Auflage der Sprachbeschreibung **Programmieren in C** von Brian W. Kernighan und Dennis M. Ritchies und war lange quasi der „Standard“ für C. Diese Sprachbeschreibung war aber weder vollständig noch exakt.

1989 wurde **ANSI-C** (C89, ANSI X3.159:1989) veröffentlicht. Das Resultat der Arbeit einer 1983 von ANSI (*American National Standards Institute*) eingerichteten Kommission, die eine umfassende Definition von C erstellen sollte.

1990 wurde ANSI-C inhaltlich unverändert von der ISO (*International Standards Organization*) übernommen (C90, ISO/TEC 9899:1990).

1994 erste Ergänzungen zum ANSI-Standard (ISO C Amendment 1).

Geschichte (2/2)

1999 Überarbeitung des Standards (C99, ISO/TEC 9899:1999), wobei auf Abwärtskompatibilität geachtet wurde.

2011 erneute Überarbeitung (**C11**, ISO/IEC 9899:2011), wobei viele von C99 geforderte Ergänzungen nun optional sind.

2017/2018 beheben von Fehler in C11, keine Einführung neuer Sprachfunktionen (ISO/IEC 9899:2018).

ANSI-C

Der Standard von 1989 beruht in weiten Teilen auf der ursprünglichen Sprachbeschreibung von Kernighan/Ritchie, der **Standardcompiler** verhält sich im wesentlichen wie dort beschrieben.

Wichtige Neuerungen.

- Die Syntax zur Deklaration und Definition von Funktionen wurde so erweitert, dass der Compiler eventuelle Widersprüche zwischen den Parametern in Deklaration und Definition einer Funktion und den Argumenten in ihren Aufrufen erkennen **kann**.
- Der Standard legt fest, welchen Umfang die **Standardbibliothek** haben **muss**. Diese Bibliothek enthält Funktionen zur Ein-/Ausgabe, zum Zugriff auf das Betriebssystem, zur Manipulation von Zeichenketten, usw.

Ein Programm, das zum Zugriff auf Betriebssystem und/oder Hardware des Rechners ausschließlich Funktionen aus dieser Bibliothek verwendet, ist **portabel**. D.h. es lässt sich auf jedem beliebigen Rechner, auf dem ein Standardcompiler mit Standardbibliothek zur Verfügung steht, compilieren.

- Überarbeitung der Regeln für numerisches Rechnen.

Wichtige Neuerungen in C99.

- Ganzzahliger Datentyp `long long` (mindestens 64 Bit)
- Lokale Felder variabler Größe (sogenannte *Variable Length Arrays*) (optional seit C11)
- Zeilenkommentare `//`
- Boolescher Datentyp `_Bool`, Header `<stdbool.h>`
- Erweiterte Unterstützung von Gleitkommazahlen inklusive neuer mathematischer Funktionen in der C-Bibliothek.
- Alias-freie Zeiger (Schlüsselwort `restrict`)
- Frei platzierbare Deklaration von Bezeichnern
- Verbot des *impliziten int* (implizite Funktionsdeklarationen)
- Präprozessor-Makros mit variabler Parameteranzahl
- Datentyp `_Complex` für komplexe Zahlen

Wichtige Neuerungen in C11.

- Transparente Unions und Structs
- Exklusiver Dateizugriff
- Zusicherungen (*assertions*) während der Compile-Zeit
- Funktionen ohne Wiederkehr, Schlüsselwort `_Noreturn`
- Funktion `gets` entfällt
- Atomare Operatoren (optional)
- Thread-Unterstützung (optional)

Einleitung

Programmiersprachen

Schritte der Programmentwicklung

Der Zeichensatz von C

Der Aufbau von C-Quellcode

Die Geschichte von C

Hello World

Hello World

```
1  /*****
2   * hello world
3   *****/
4  #include <stdio.h>                                // preprocessor directive
5
6  // = main function =====
7  int main() {
8      printf("Hello World!\n");                      // text output
9      return 0;                                       // exit
10 }
```

Hello World, Kommentare

```
1  /*****
2   * hello world
3   *****/
4  #include <stdio.h>                                // preprocessor directive
5
6  // = main function =====
7  int main() {
8      printf("Hello_World!\n");                      // text output
9      return 0;                                       // exit
10 }
```

Kommentare werden vom Compiler ignoriert.

Blockkommentare

- beginnen mit `/*` und enden mit `*/`.
- können **nicht geschachtelt** werden, denn das erste Auftreten von `*/` beendet den Kommentar, egal wie oft vorher `/*` gelesen wurde.

Zeilenkommentare

- beginnen mit zwei Schrägstrichen `//` und kommentieren den Rest einer Zeile aus, d.h. der Kommentar gilt von diesen Zeichen bis zum Ende der Zeile (Zeilenumbruchzeichen).

Kommentarblöcke erstrecken sich über die ganze Breite der Zeilen.

- Jede Quelldatei sollte mit einem Block beginnen, der Inhalt, Autor und das Datum der letzten Änderung enthält.
- Jede Funktion sollte durch einen eigenen Kommentarblock eingeleitet werden.
- Für die Umrandungen von Kommentarblöcke sollten für unterschiedliche Zugehörigkeit Zeichen mit unterschiedlichem Schwärzungsgrad verwendet werden.
 - ▶ `/*****` für den Block am Programmanfang
 - ▶ `/*=====` für die Blöcke vor Funktionen
 - ▶ `/*----*/` für Blöcke im Programmtext
 - ▶ usw.

Inline-Kommentare zur Erläuterung des Ablaufs.

- Nicht jede einzelne Zeile kommentiert, aber auch nicht zu sparsam mit Kommentaren sein.
- Beschreiben **warum** etwas geschieht, **nicht was** geschieht.

Hello World, Präprozessordirektive

```
1  /*****
2   * hello world
3   *****/
4  #include <stdio.h>                                // preprocessor directive
5
6  //= main function =====
7  int main() {
8      printf("Hello_World!\n");                      // text output
9      return 0;                                       // exit
10 }
```

Eine Zeile wird als **Präprozessordirektive** interpretiert, wenn sie als erstes Zeichen ein **#** enthält.

Der **Präprozessor** ist ein Programm, das den Programmcode vor dem eigentlichen Compiler durchläuft und Textersetzungen vornimmt.

Die Direktive `#include <stdio.h>` besagt, dass der Inhalt der Datei `stdio.h` an dieser Stelle in den Programmtext eingefügt werden soll.

Hello World, Hauptprogramm `main`

```
1  /*****
2   * hello world
3   *****/
4  #include <stdio.h>                                // preprocessor directive
5
6  // = main function =====
7  int main() {
8      printf("Hello_World!\n");                      // text output
9      return 0;                                       // exit
10 }
```

Der **Typ des Funktionswertes** steht vor dem Funktionsnamen.

Die **Parameterliste**, in runde Klammern eingeschlossen, ermöglicht die Kommunikation zwischen einer Funktion und ihrer Umwelt.

Standard

- `main` hat den ganzzahligen Typ `int` als Funktionswert.
- `void` entspricht leerer Parameterliste.

Hauptprogramm `main`

Achtung. Im Skript finden Sie `main(void)` statt `main()`.

Das explizite Kennzeichnen einer leeren Parameterliste mit `void` entspricht dem ANSI-Standard, ist aber seit C99 nicht mehr nötig.

Hingegen wäre das Weglassen des Funktionstyps wäre bei ANSI-C erlaubt, aber seit C99 ergänzt der Compiler den Typ `int` nicht mehr implizit, wenn der Typ einer Funktion nicht explizit angegeben ist.

Während dieses Kurses wird der Funktionstyps immer angegeben.

Hello World, Rumpf einer Funktion

```
1  /*****
2   * hello world
3   *****/
4  #include <stdio.h>                                // preprocessor directive
5
6  // = main function =====
7  int main() {
8      printf("Hello_World!\n");                    // text output
9      return 0;                                    // exit
10 }
```

Der Rumpf einer Funktion wird in geschweifte Klammern eingeschlossen.

Der Rumpf enthält die Anweisungen, die beim Aufruf der Funktion ausgeführt werden. Jede Anweisung wird durch ein Semikolon ; abgeschlossen.

Um die Lesbarkeit von Programmen zu verbessern, ist es üblich die Anweisungen innerhalb eines Rumpfes einzurücken.

Hello World, Bibliotheksfunktion

```
1  /*****
2   * hello world
3   *****/
4  #include <stdio.h>                                // preprocessor directive
5
6  // = main function =====
7  int main() {
8      printf("Hello_World!\n");                      // text output
9      return 0;                                       // exit
10 }
```

Der Aufruf der Bibliotheksfunktion **printf** schreibt die in Anführungszeichen stehende Zeichenkette auf die Standardausgabe (den Bildschirm).

Die Zeichenkombination **\n** ist **ein** Zeichen und bewirkt bei ihrer Ausgabe einen Zeilenumbruch (newline-Zeichen).

Der Compiler erwartet öffnende und schließende Anführungszeichen in derselben Zeile.

Hello World, beenden

```
1  /*****
2   * hello world
3   *****/
4  #include <stdio.h>                                // preprocessor directive
5
6  // = main function =====
7  int main() {
8      printf("Hello_World!\n");                    // text output
9      return 0;                                    // exit
10 }
```

Die **return**-Anweisung beendet eine Funktion und liefert den nachstehenden Wert (hier 0) als Funktionswert an die rufende Funktion zurück.

Für **main** ist die rufende Funktion das Betriebssystem.

Hello World, übersetzen und ausführen (ANSI-C)

```
1  /*****
2   * hello world
3   *****/
4  #include <stdio.h>                                /* preprocessor directive */
5
6  /*= main function =====*/
7  int main(void) {
8      printf("Hello_World!\n");                      /* text output */
9      return 0;                                       /* exit      */
10 }
```

Achtung. Keine Zeilenkommentare (//) in ANSI-C.

Dem Compiler (gcc) kann mitgeteilt werden, dass alle Warnungen (-Wall) ausgegeben werden, der Quelltext dem ANSI-Standard genügen (-ansi) und das andere Quelltexte zurückgewiesen werden (-pedantic) sollen.

```
> gcc -Wall -ansi -pedantic hello_world.c
> ./a.out
Hello World!
```

Hello World, übersetzen und ausführen (C99)

```
1  /*****
2   * hello world
3   *****/
4  #include <stdio.h>                                /* preprocessor directive */
5
6  /*= main function =====*/
7  int main(void) {
8      printf("Hello_World!\n");                      /* text output */
9      return 0;                                       /* exit */
10 }
```

Mit `-std=...` kann man den Compiler (gcc) anweisen einen anderen Standard zu verwenden, z.B. C99.

```
> gcc -std=c99 hello_world.c
> ./a.out
Hello World!
```

Hello World, übersetzen und ausführen (C11)

```
1  /*****
2   * hello world
3   *****/
4  #include <stdio.h>                                // preprocessor directive
5
6  //= main function =====
7  int main() {
8      printf("Hello_World!\n");                      // text output
9      return 0;                                       // exit
10 }
```

Während dieses Kurses wird immer der Standard c11 verwendet, dass ist aktuell die *default*-Einstellung von gcc.

Man kann dem ausführbaren Programm auch einen (aussagekräftigen) Namen geben (`-o name`).

```
> gcc -o hello hello_world.c
> ./hello
Hello World!
```
